

이음 프로젝트의

SW 설계 기술서(SIP)

2026. 05. 03

팀장 정승민
팀원 김수빈
김재현
최정원

이음

SW 구현 계획서(SIP)

목차

1. 개요

- 1.1 목적 및 범위
- 1.2 개발 환경

2. 핵심 용어 정의 및 약어

- 2.1 용어
- 2.2 약어

3. 참고 문헌

4. 컴포넌트 명세

- 2.1 컴포넌트 구조도
- 2.2 컴포넌트 다이어그램

5. 빌드 계획

6. 업무 분류

7. 구현 일정 및 기준

- 7.1 구현 일정
- 7.2 구현 기준

8. 검사 일정

9. 추적성 검사 매트릭스

1. 개요

1.1 목적 및 범위

본 문서는 동네 기반 통합 거래 플랫폼 이음의 소프트웨어 구현 단계에서 수행할 작업과 일정, 빌드 환경, 업무 분장, 검사 기준을 정의한다. 앞서 작성된 SOP, PC, PMP, SRD, SRS, SDD에 근거하여 실제 코드 구현, 빌드, 통합, 단위 검증까지의 활동을 체계적으로 관리하는 것을 목적으로 한다.

본 계획서의 적용 범위는 다음과 같다:

- 프론트엔드(React Native 모바일 앱, React 웹) 및 백엔드(Spring Boot) 코드 구현
- DB 스키마 적용 및 Redis 설정
- 외부 시스템 연동(PortOne, FCM, AWS S3, OAuth)
- 단위 테스트(Unit Test) 작성 및 검증
- 빌드 및 배포 자동화(CI/CD) 환경 구축

1.2 개발 환경

(1) 하드웨어 환경

구분	사양
개발 PC	i7 / M2 이상, RAM 16GB
서버	AWS EC2 t2.micro
DB	AWS RDS MySQL
캐시	Redis (Docker)
스토리지	AWS S3

(2) 소프트웨어 환경

분류	도구/버전
백엔드 언어	Java 17
백엔드 프레임워크	Spring Boot 3.2.x
ORM	Spring Data JPA + QueryDSL 5.x
DB	MySQL 8.0
캐시	Redis 7.x (Docker)
모바일 앱	React Native 0.73 + JavaScript 5.x
웹	React 18 + JavaScript 5.x + Vite
빌드 도구	Gradle 8.x (백엔드), npm/yarn (프론트엔드)
형상 관리	Git, GitHub
이슈 관리	GitHub Issues + Projects
CI/CD	GitHub Actions
컨테이너	Docker, Docker Compose
IDE	IntelliJ IDEA, VS Code
API 문서화	Swagger (springdoc-openapi 2.x)
디자인 도구	Figma
협업 도구	Notion, Slack, Zoom

(3) 외부 연동 시스템

- PortOne (결제 PG)
- FCM (Firebase Cloud Messaging — 푸시 알림)
- AWS S3 (이미지 저장)
- Kakao OAuth, Naver OAuth (소셜 로그인)
- KakaoMap API (지도 / 좌표 변환)
- 사업자번호 검증 API (국세청)

2. 핵심 용어 정의 및 약어

2.1 용어

용어	정의
이음(Eeum)	본 프로젝트의 명칭으로, 동네 기반 사용자-상점 통합 거래 플랫폼
회원(Account)	이음 서비스에 가입한 일반 사용자
사장 회원(Owner)	동네 상점을 운영하는 사업자 회원으로, 관리자 승인 후 활동 가능한 회원
활동 지역(AccountRegion)	GPS 인증을 통해 등록한 사용자의 활동 지역. 최대 2개까지 등록 가능
대표 지역(Primary Region)	사용자의 주요 활동 지역으로 설정된 지역
상점(Store)	사장 회원이 운영하는 동네 상점. 사장 회원과 1:1로 매핑
이벤트 상품(EventProduct)	한정 수량 또는 한정 시간 동안 할인 판매되는 상품
매장 방문 예약(VISIT)	사용자가 매장 방문 시간을 미리 예약하는 기능
상품 예약(PRODUCT)	결제 전 일정 시간 동안 상품 재고를 임시 확보하는 예약
중고거래(UsedProduct)	사용자 간 개인 중고 물품 거래 기능
동네 커뮤니티(Community)	활동 지역 기반의 게시글, 댓글, 대댓글 커뮤니티
멱등성(Idempotency)	동일한 요청이 중복 도달하더라도 결과가 동일하게 보장되는 성질
Soft Delete	데이터를 즉시 삭제하지 않고 deletedAt을 기록한 뒤 일정 기간 보존하는 삭제 정책
Hard Delete	데이터를 즉시 물리적으로 삭제하는 정책
Polymorphic 참조	refType + refId 조합으로 여러 도메인의 대상을 참조하는 패턴
협업 도구	Notion, Slack, Zoom

2.2 약어

약어	원어
SOP	Statement of Problem
PC	Project Charter
PMP	Project Management Plan
SRD	Software Requirements Definition
SRS	Software Requirements Specification
SDD	Software Design Document
SIP	Software Implementation Plan
STP	Software Test Plan
STR	Software Test Report
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ORM	Object-Relational Mapping
JPA	Java Persistence API
DTO	Data Transfer Object
AOP	Aspect-Oriented Programming
CQRS	Command Query Responsibility Segregation
CASCADE	연관된 데이터 자동 삭제
TPS	Transactions Per Second
FCM	Firebase Cloud Messaging
OAuth	Open Authorization
CI/CD	Continuous Integration / Continuous Deployment
WBS	Work Breakdown Structure
CPM	Critical Path Method

3. 참고 문헌

번호	원어	출처
1	이음 상세 문제 기술서 (SOP)	-
2	이음 프로젝트 현장 (PC)	-
3	이음 프로젝트 관리 계획서 (PMP)	-
4	이음 SW 요구사항 정의서 (SRD)	-
5	이음 SW 요구사항 명세서 (SRS)	-
6	이음 SW 설계 문서 (SDD)	-
7	IEEE Std 1016-2009 — Software Design Descriptions	-
8	IEEE Std 829-2008 — Software and System Test Documentation	-
9	Spring Boot 공식 문서	https://spring.io/projects/spring-boot
10	React Native 공식 문서	https://reactnative.dev
11	PortOne API 문서	https://developers.portone.io
12	AWS 공식 문서	https://docs.aws.amazon.com

4. 컴포넌트 명세

4.1 컴포넌트 구조도

이름 시스템은 계층(Layer) + 도메인(Bounded Context) 구조의 Spring Boot 단일 애플리케이션으로 구성된다. 클라이언트는 React Native 모바일 앱과 React 웹으로 구현되며, 동일한 REST API를 호출한다.

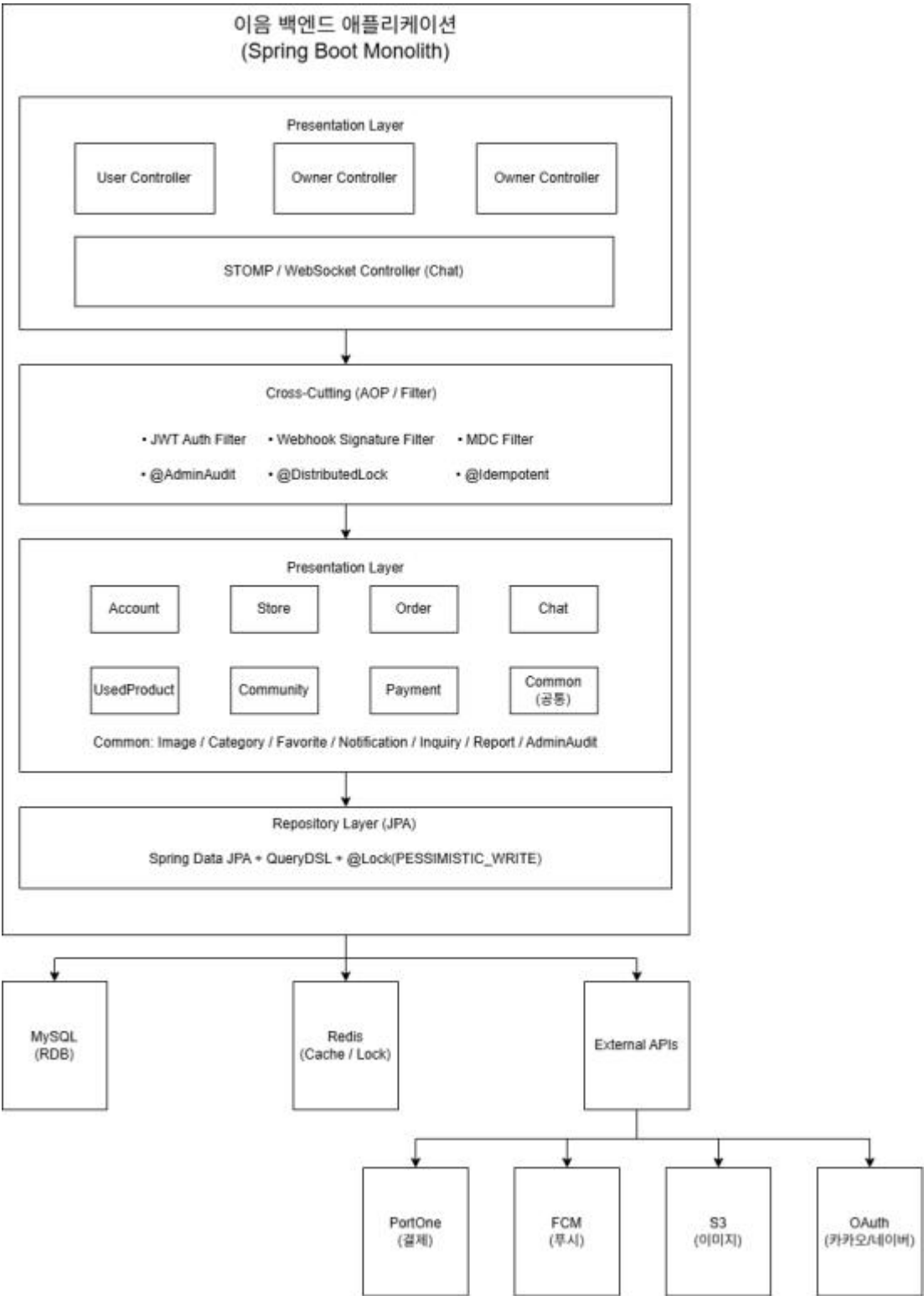
계층별 책임

- **Presentation Layer:** HTTP/WebSocket 요청 수신 및 응답 직렬화
- **AOP / Filter Layer:** 인증, 감사 로그, 분산 락, 멍등성 등 횡단 관심사 처리
- **Service Layer:** 비즈니스 로직과 트랜잭션 경계
- **Repository Layer:** JPA + QueryDSL 기반 데이터 접근
- **Infrastructure:** MySQL, Redis, 외부 API

도메인 구성

- Account, Store, Order/Payment, UsedProduct, Community, Chat
- 공통 도메인: Image, Category, Favorite, Notification, Inquiry, Report, AdminAudit

4.2 컴포넌트 다이어그램



도메인별 컴포넌트 책임

도메인	핵심 책임
Account	회원가입, 로그인, 활동 지역, 사장 정보, 토큰 관리, 감사 로그
Store	매장 등록/관리, 상품/이벤트 상품, 리뷰/답글, 매장 공지, 상품 옵션
Order / Payment	주문, 결제(PortOne), 예약, 장바구니, 환불
UsedProduct	중고상품 등록/거래, 거래 리뷰
Community	게시글, 댓글, 대댓글, 좋아요
Chat	채팅방, 메시지(WebSocket), 참여자 관리
Common	이미지(S3), 카테고리, 찜, 알림, 문의, 신고

5. 빌드 계획

5.1 빌드 환경

이름 시스템은 GitHub Actions 기반의 CI/CD 파이프라인을 구축하여 코드 변경 시 자동 빌드 및 테스트가 실행되도록 한다.

항목	내용
형상 관리	GitHub 기반 Git 형상 관리
브랜치 전략	main, develop, feature/*, hotfix/*
빌드 도구	Gradle 8.x (백엔드), npm/yarn (프론트엔드)
CI/CD 도구	GitHub Actions
컨테이너	Docker, Docker Compose
빌드 트리거	PR 생성 시, develop 또는 main 머지 시
배포 환경	AWS EC2 단일 인스턴스 기반 프리티어 환경

5.2 브랜치 전략

브랜치	용도
main	프로덕션 배포용 브랜치. 보호 브랜치로 관리
develop	기능 통합 및 테스트용 개발 브랜치
feature/{도메인}-{기능}	기능 단위 작업 브랜치. 예: feature/order-create
hotfix/{이슈번호}	운영 또는 주요 오류에 대한 긴급 수정 브랜치

5.3 빌드 단계

1. 코드 체크아웃 (GitHub Actions)
2. 의존성 캐싱 (Gradle / npm)
3. 정적 분석 (SonarLint, ESLint, Checkstyle)
4. 단위 테스트 실행 (JUnit5, Jest)
5. 통합 테스트 실행 (Testcontainers + MySQL/Redis)
6. 빌드 산출물 생성 (.jar / .apk / Web 정적 파일)
7. Docker 이미지 빌드
8. EC2 배포 (SSH + docker-compose pull/up)
9. Health Check (/actuator/health)

5.4 배포 전략

- Blue-Green 배포 미적용: 단일 EC2 인스턴스 환경이므로 무중단 배포는 향후 트래픽 증가 시 도입
- Rolling Update: 단일 인스턴스에서는 docker-compose down → up 으로 짧은 다운타임 허용 (수 초)
- 롤백 전략: 직전 이미지 태그를 보존하여 즉시 재배포 가능 (eeum:previous)

6. 업무 분류

6.1 팀 구성 및 역할

역할	담당자	책임
팀장 / Backend	정승민	전체 일정 관리, 문서화, 백엔드 도메인, 코드 리뷰, 배포
Backend	김수빈	백엔드 도메인, UI, 코드 리뷰
Frontend Web / Mobile	김재현	React Web, React Native, 코드 리뷰
Frontend Mobile	최정원	React Native, 코드 리뷰

7. 구현 일정 및 기준

7.1 구현 일정

총 구현 기간: 2026.05.05 ~ 2026.06.23 (약 7주)

Phase	기간	주요 활동
Phase 1: 기반 구축	2026.05.05 ~ 2026.05.11 (1주)	프로젝트 셋업, DB 스키마, 공통 모듈, BaseEntity, 예외 처리, JWT 기반 구성
Phase 2: Account / Auth	2026.05.12 ~ 2026.05.18 (1주)	회원가입, 로그인, OAuth, 활동 지역, 사장 정보 구현
Phase 3: Store / Product	2026.05.19 ~ 2026.05.28 (1.5주)	매장, 상품, 옵션, 이벤트 상품, 리뷰, 매장 공지 구현
Phase 4: Order / Payment	2026.05.28 ~ 2026.06.06 (1.5주)	장바구니, 주문, PortOne 결제, 예약, 환불 구현
Phase 5: 부가 도메인	2026.06.07 ~ 2026.06.13 (1주)	UsedProduct, Community, Chat, Notification 구현
Phase 6: 통합 / 안정화	2026.06.14 ~ 2026.06.23 (1.5주)	관리자 기능, 통합 테스트, 버그 수정, 배포

7.2 구현 기준

1. 코딩 표준

- Java: Google Java Style Guide 준수 + Checkstyle 적용
- TypeScript: ESLint (Airbnb 스타일) + Prettier
- 명명 규칙:
 - Entity 필드: camelCase (예: accountId, createdAt)
 - DB 컬럼: snake_case (예: account_id, created_at)
 - URL: kebab-case (예: /used-products)
 - 메서드: camelCase, 동사로 시작

2. 설계 원칙 준수

- 모든 Entity는 BaseEntity 상속 (createdAt/modifiedAt 자동 관리)
- Repository 파라미터는 명시적 타입 표기
- Polymorphic 참조는 도메인별 RefType ENUM + refId 패턴
- 가격 필드는 BigDecimal 사용
- Hard Delete 기본, Account/ChatMessage/Category만 Soft Delete

3. 단위 테스트 기준

- 커버리지 목표: Service Layer 70% 이상, Controller Layer 60% 이상
- 테스트 도구: JUnit 5 + Mockito + AssertJ
- 테스트 작성 원칙: Given-When-Then 구조, 메서드명은 한글 표현 가능
- 외부 의존성: 단위 테스트에서는 모두 Mocking, 통합 테스트에서만 실제 사용

4. 코드 리뷰 기준

- 모든 PR은 최소 1명의 리뷰 승인 후 머지
- 주요 변경(공통 모듈, DB 스키마)은 두 사람 모두 승인 필수
- PR 템플릿 사용: 변경 내용, 테스트 방법, 영향 범위 명시

8. 검사 일정

검사 항목	시점	담당	산출물
코드 정적 분석	매 PR	GitHub Actions	SonarLint / ESLint 리포트
단위 테스트 검증	매 PR	GitHub Actions	JUnit / Jest 리포트
통합 테스트 검증	주 1회 금요일	팀 전체	Testcontainers 결과
Phase 별 검토	각 Phase 종료일	팀 전체	회고 노트
기능별 동작 확인	Phase 4 이후	팀 전체	기능 체크리스트
통합 빌드 / 배포 검증	2026.06.13	팀 전체	배포 로그, Health Check 결과
최종 통합 검사	2026.06.20	팀 전체	통합 검사 결과서

9. 추적성 매트릭스

요구사항 ID	요구사항	SDD 매핑	구현 모듈
FR-AUTH-001	이메일 회원가입	4.1 Account 도메인 / 7.2 AuthService	AuthService.signup()
FR-AUTH-002	OAuth 소셜 로그인	7.2 AuthService.oauthLogin	AuthService.oauthLogin()
FR-AUTH-003	JWT 발급 / 재발급	6.10 보안 패턴	JwtProvider, TokenRepository
FR-AUTH-004	비밀번호 재설정	7.2 AuthService.resetPassword	AuthService.resetPassword()
FR-ACC-001	활동 지역 등록(GPS 인증)	7.1 AccountRegionService	AccountRegionService.registerRegion()
FR-ACC-002	회원 탈퇴(30일 유예)	6.6 Soft Delete 정책	AccountService.withdraw()
FR-STORE-001	사장 매장 등록	7.3 OwnerStoreService	OwnerStoreService.register()
FR-STORE-002	상품 등록(옵션 포함)	7.3 OwnerProductService	OwnerProductService.register()
FR-STORE-003	이벤트 상품 등록 / 관리	7.3 OwnerEventProductService	OwnerEventProductService
FR-STORE-004	매장 공지 등록 / 관리	7.3 StoreNoticeService	StoreNoticeService
FR-STORE-005	매장 검색	7.3 StoreService.searchStores	StoreRepositoryCustom
FR-ORD-001	장바구니	7.4 CartService	CartService
FR-ORD-002	주문 생성 및 결제	7.4 OrderService.createOrder	OrderService.createOrder()
FR-ORD-003	PortOne Webhook 처리	6.9 외부 시스템 연동	PaymentService.handleWebhook()
FR-ORD-004	환불 처리	7.4 OrderService.cancel	OrderService.cancel()
FR-ORD-005	매장 방문 예약	7.4 ReservationService	ReservationService

FR-USED-001	중고상품 등록	7.5 UsedProductService	UsedProductService
FR-USED-002	거래 완료 처리	7.5 UsedProductService.completeSale	UsedProductService.completeSale()
FR-COM-001	게시글 작성	7.6 CommunityPostService	CommunityPostService
FR-COM-002	댓글 / 대댓글 작성	7.6 CommunityCommentService	CommunityCommentService
FR-COM-003	좋아요	7.6 PostLikeService	PostLikeService
FR-CHAT-001	1:1 채팅방 생성(먹등)	7.7 ChatRoomService	ChatRoomService
FR-CHAT-002	메시지 발송(WebSocket)	7.7 ChatMessageService	ChatMessageService
FR-NOTI-001	알림 발송(이벤트 기반)	6.2 이벤트 아키텍처	NotificationService
FR-FAV-001	찜 토글	7.8 FavoriteService	FavoriteService
FR-IMG-001	이미지 업로드(Presigned URL)	6.9 외부 시스템 연동	ImageService
FR-INQ-001	문의 등록 / 답변	7.8 InquiryService	InquiryService
FR-REP-001	신고 등록	7.8 ReportService	ReportService
FR-ADMIN-001	사장 승인(감사 로그)	7.9 AdminAccountService	AdminAccountService
NFR-PERF-001	API 응답 p95 < 500ms	6.4 캐싱 / 6.11 페이징	Redis 캐시 + Slice 페이징
NFR-SEC-001	JWT 인증 / 권한 분리	6.10 보안 패턴	SecurityConfig
NFR-AVAIL-001	가용성 99% 이상	6.14 모니터링	Actuator + Health Check